DEVELOPING CODE · DOMAIN 1 - DEVELOPING CODE FOR DATA PROCESSING

Jobs, tasks and control flow

Section 1. Dependencies, branching, loops, retries, and the at-most-once problem.

SECTION 1 of 10 WEIGHT 22% TOPIC Jobs

admin@sthithapragna.com

That last objective is **almost word for word** the section headings of one documentation page. When a guide gets that specific, it is telling you where to look.

Three concepts, and they nest

CONCEPT	WHAT IT IS
Job	The unit you schedule and run. Holds tasks, triggers and parameters.
Task	One piece of work. A notebook, a pipeline, a wheel, a SQL query, another job.
Trigger	What starts the job. A schedule, a file arrival, a continuous run, or manual.

ON THE EXAM

Tasks form a directed acyclic graph. Databricks runs upstream tasks first, and runs as many in parallel as the graph allows.

The six run-if conditions

CONDITION	WHEN THE TASK RUNS
All succeeded	Every dependency ran and succeeded. This is the default.
At least one succeeded	One or more dependencies succeeded.
None failed	No dependency failed. Skipped ones are acceptable.
All done	Every dependency finished, whatever the outcome.
At least one failed	Used for error handling branches.
All failed	Used when everything upstream has gone wrong.

ON THE EXAM

When All succeeded is unmet, the downstream task is marked Upstream failed rather than simply skipped. That distinction shows up in monitoring.

The If/else condition task

- Runs part of the graph based on a **boolean expression**, rather than on whether something failed.
- The condition can read **task values**, **job parameters**, or dynamic values.
- Downstream tasks default to the **true** branch. The false branch must be selected explicitly.
- **It fails if the upstream task supplying its condition value is disabled.**

> ON THE EXAM
>
> Run if branches on outcome. If/else branches on value. A question about 'only if new data arrived' is asking for If/else.

Task values carry state between tasks

- A task can **set a value** that later tasks read. That is how a computation's result leaves a task.
- They are what makes a meaningful If/else condition possible. Count the rows, then branch on the count.
- They can also be passed as parameters into downstream tasks.
- They are not a general message bus. They carry small results, not data.

> ON THE EXAM
>
> Without task values, an If/else can only branch on things known before the job started. With them, it can branch on what actually happened.

The For each task needs two parts

- It runs **another task** in a loop, passing different parameters to each iteration.
- You must define **both** the For each task **and a nested task**. One alone does nothing.
- The nested task is an ordinary task type. There is nothing special about it.
- The input array can be written by hand, or generated dynamically from an earlier task.

> ON THE EXAM
>
> Dynamically generated arrays are the powerful case: one task discovers which regions have new files, and the loop processes exactly those.

The default depends on the configuration

CONFIGURATION	RETRY BEHAVIOUR
Most job configurations	No retries on failure. This is the default.
Serverless jobs	Auto-optimisation retries failed tasks by default.
Continuous trigger mode	Automatic retries with exponential backoff.

ON THE EXAM

You cannot set a retry policy on a continuous job. Its retry behaviour is system-defined and not configurable.

Some features assume you retry

- **Schema evolution with Structured Streaming assumes retries are configured.**
- When the schema changes, the stream stops. The **retry is what restarts it** with the new schema.
- Without retries, that stream simply stays failed until somebody notices.
- So a retry policy is not only about transient errors. It is a required part of some designs.

> ON THE EXAM
>
> This is the reason a question about a stream that stops on every schema change often has 'configure retries' as its answer.

Serverless auto-optimisation picks your compute **and retries your failed tasks**. Both halves are on by default.

At least once against at most once

LEAVE IT ENABLED

At least once

- Databricks' recommendation
- Transient failures resolve themselves
- Critical workloads do complete
- A task may run more than once

DISABLE IT

At most once

- For workloads that are not idempotent
- A payment, an email, an external API call
- Uncheck it in the Retry Policy dialog
- In configuration: disable auto optimization

STATED IN THE DOCS

The docs recommend leaving it on so critical workloads run at least once. The exception they name is jobs that are not idempotent.

High memory is notebook tasks only

- You can configure a **higher memory size** for serverless notebook workloads.
- **Only for notebook task types.** Not for wheel tasks, not for pipelines.
- Set through the **Memory setting in the notebook's Environment side panel**, not in the job.
- It carries a **higher DBU emission rate** than standard memory, so it is not free.

> ON THE EXAM
>
> Note where the setting lives. It is configured on the notebook environment, which is also where dependencies are declared.

Environments and dependencies on a task

- A serverless task references an **environment by key**. The environment carries a version and dependencies.
- Several tasks can share one environment definition, so a change lands in one place.
- A **notebook task cannot take a task environment spec**. Its dependencies come from the notebook environment panel.
- Which is why the memory setting and the dependency list live together.

> ON THE EXAM
>
> This is the module 3 material, seen from the job side. The attachment point differs by task type, and notebook tasks are the exception.

What you no longer control

- **Autoscaling and Photon are enabled automatically.** There is no switch.
- Spark configuration can be set at **session level only**, from a notebook, and only from an allowed list.
- **Egress traffic cannot be controlled**, so serverless jobs have full internet access.
- A **Performance optimized** setting on the job affects only its serverless tasks.

> ON THE EXAM
>
> The egress point is a security consideration, not a performance one. It can rule serverless out on its own.

Creating and automating jobs

INTERFACE	USE IT FOR
UI	Authoring and debugging. The visual graph is genuinely useful for branching.
REST API	Programmatic creation, triggering from outside, and monitoring.
CLI	Scripting and automation from a terminal or a delivery pipeline.
Bundles	Defining the job as version-controlled configuration. The production answer.

ON THE EXAM

The objective names UI, API and CLI. In practice bundles wrap the CLI, which is how a job definition gets into version control.

What to carry into the exam

- **Run if** branches on outcome. **If/else** branches on a value, usually a task value.
- **For each needs a nested task.** The array can be generated dynamically.
- Default is **no retries**, except on serverless and in continuous mode. Continuous cannot be configured.
- Disable serverless auto-optimisation for **at most once** work. Leave it on otherwise.
- **High memory is notebook tasks only**, set in the notebook environment panel.

> ON THE EXAM
>
> That completes Section 1, the largest section on the exam at 22%. Section 2 is next, and it is much smaller.

A job has an ingestion task followed by three expensive transformation tasks. On most days no new files arrive, and the transformations run anyway, producing identical output at significant cost.

The team wants the transformations to run only when ingestion actually loaded new records.

What is the right construction?

A Set the transformations' run-if condition to At least one succeeded.

B Have ingestion set a task value with the row count, then use an If/else task to branch on it.

C Configure the ingestion task to fail when no new files arrive, so downstream tasks are skipped.

D Add a For each task over the ingested files, so transformations run once per file.

◆ PAUSE AND DECIDE

What is the right construction?

A Set the transformations' run-if condition to At least one succeeded.

Run if branches on outcome. Ingestion succeeds whether or not it found anything.

B Have ingestion set a task value with the row count, then use an If/else task to branch on it.

Branching on a value computed by an earlier task is exactly what task values and If/else exist for.

C Configure the ingestion task to fail when no new files arrive, so downstream tasks are skipped.

Makes a normal day look like an incident, and pollutes alerting and run history.

D Add a For each task over the ingested files, so transformations run once per file.

Solves a different problem, and changes the transformation from one run to many.

ON THE EXAM

Outcome or value is the first question. Nothing failed here, so run-if conditions cannot express what is wanted.

A serverless job task calls a third-party payment provider's API to issue refunds. During a network incident the task failed, and the team later found that a number of customers had been refunded twice.

The task itself has no retry policy configured, and the team is confident their code runs once per invocation.

What most likely happened, and what should they change?

A Serverless auto-optimisation retried the failed task. Disable auto-optimisation for this task.

B The job was in continuous mode, which retries with exponential backoff. Switch to a scheduled trigger.

C Nothing retried it. The provider's API must have duplicated the requests.

D The default retry policy is three attempts. Set max retries to zero.

◆ PAUSE AND DECIDE

What most likely happened, and what should they change?

A Serverless auto-optimisation retried the failed task. Disable auto-optimisation for this task.

On serverless, auto-optimisation retries failed tasks by default. No retry policy is needed for a retry to happen.

B The job was in continuous mode, which retries with exponential backoff. Switch to a scheduled trigger.

Nothing describes a continuous job, and refunds are not a continuous workload.

C Nothing retried it. The provider's API must have duplicated the requests.

Blames the third party for documented default behaviour on this platform.

D The default retry policy is three attempts. Set max retries to zero.

There is no default of three, and setting max retries does not disable auto-optimisation.

ON THE EXAM

This is the exact case the docs name: a workload that must run at most once because it is not idempotent.

> A nightly job must process files for whichever of 40 country codes received new data that day. The list changes daily. Currently there are 40 hard-coded tasks, most of which exit immediately.
>
> The team wants one construct that processes only the countries with new data.

What should they build?

A An If/else task per country, each branching on whether that country has data.

B A task that produces the list of active countries, then a For each task with a nested processing task.

C A single task that loops over countries inside the notebook code.

D A For each task alone, with the country list as its input array.

◆ PAUSE AND DECIDE

What should they build?

A An If/else task per country, each branching on whether that country has data.

Replaces 40 tasks with 80. The count is the problem, not the exit behaviour.

B A task that produces the list of active countries, then a For each task with a nested processing task.

Dynamically generated input array plus a nested task. Exactly what For each is for.

C A single task that loops over countries inside the notebook code.

Works, and hides the loop from the job graph, so per-country failures and retries disappear.

D A For each task alone, with the country list as its input array.

A For each task with no nested task has nothing to run.

ON THE EXAM

D is the structural trap. A For each task is half a construction; without a nested task it does nothing.

> A serverless notebook task fails intermittently with out-of-memory errors on the largest partitions. The logic cannot easily be restructured before a deadline. A colleague suggests configuring high memory.
>
> A second task in the same job is a Python wheel task that has the same problem.

What is true about this suggestion?

A. High memory can be applied to both tasks in the job settings.

B. High memory applies to the notebook task only, set in its environment panel, and bills at a higher rate.

C. High memory is a cluster policy setting and does not apply to serverless.

D. High memory is enabled automatically by auto-optimisation, so no change is needed.

◆ PAUSE AND DECIDE

What is true about this suggestion?

A. High memory can be applied to both tasks in the job settings.

Wrong twice: it is notebook-only, and it is configured on the notebook environment, not the job.

B. High memory applies to the notebook task only, set in its environment panel, and bills at a higher rate.

All three constraints matter here, and the wheel task will need a different remedy.

C. High memory is a cluster policy setting and does not apply to serverless.

It is specifically a serverless feature.

D. High memory is enabled automatically by auto-optimisation, so no change is needed.

Auto-optimisation selects resources, but high memory is an explicit opt-in.

ON THE EXAM

The wheel task is in the question for a reason. Half the problem has a quick fix and half does not.

> A Structured Streaming job stops every few weeks with a schema-related error whenever the upstream producer adds a column. An engineer restarts it manually and it picks up the new schema and continues without further intervention.
>
> They want this to stop requiring a human.

What is the appropriate change?

A Configure a retry policy, since schema evolution assumes automatic retries.

B Pin the schema explicitly so new columns are ignored.

C Disable schema evolution and handle new columns in a separate job.

D Move the stream to a continuous job, which never fails on schema changes.

◆ PAUSE AND DECIDE

What is the appropriate change?

A Configure a retry policy, since schema evolution assumes automatic retries.

The documented design: the stream stops on a schema change and the retry restarts it with the new schema.

B Pin the schema explicitly so new columns are ignored.

Stops the failures by discarding the new data, which is a silent loss.

C Disable schema evolution and handle new columns in a separate job.

Adds a second job to solve what a retry policy already handles.

D Move the stream to a continuous job, which never fails on schema changes.

A continuous job does retry with backoff, but the claim that it never fails is false.

ON THE EXAM

The giveaway is that a manual restart already fixes it completely. That is a retry, done by a person.

> A regulated financial team must move a batch workload to serverless jobs. Their security policy requires that all outbound network traffic from compute be restricted to an approved list of endpoints.
>
> They also want cleanup to run whether or not the main task succeeds.

Which statement correctly assesses their plan?

A. Both requirements are met: configure egress rules on the serverless environment, and use All done for cleanup.

B. Egress cannot be controlled on serverless for workflows, so that requirement blocks it. All done handles the cleanup.

C. Egress is controlled by the workspace network policy, so serverless is fine. Use At least one failed for cleanup.

D. Both requirements are unmet: serverless has no egress control and no way to run a task unconditionally.

◆ PAUSE AND DECIDE

Which statement correctly assesses their plan?

A Both requirements are met: configure egress rules on the serverless environment, and use All done for cleanup.

The cleanup half is right. Serverless for workflows does not support controlling egress traffic.

B Egress cannot be controlled on serverless for workflows, so that requirement blocks it. All done handles the cleanup.

The docs state that jobs have full internet access because egress cannot be controlled. All done runs regardless of outcome.

C Egress is controlled by the workspace network policy, so serverless is fine. Use At least one failed for cleanup.

Wrong on both: the limitation is stated, and At least one failed skips cleanup on success.

D Both requirements are unmet: serverless has no egress control and no way to run a task unconditionally.

Half right. All done exists precisely for unconditional cleanup.

ON THE EXAM

Two requirements, one met and one not. Options that make both true or both false are the easiest to write and the easiest to spot.